

JENKINS CI/CD PROJECT

Deploy a React Vite Application to Amazon EKS using Jenkins

Name: Sneha Biswal

SIC: 24BCSI95

Silicon University

1. Project Overview

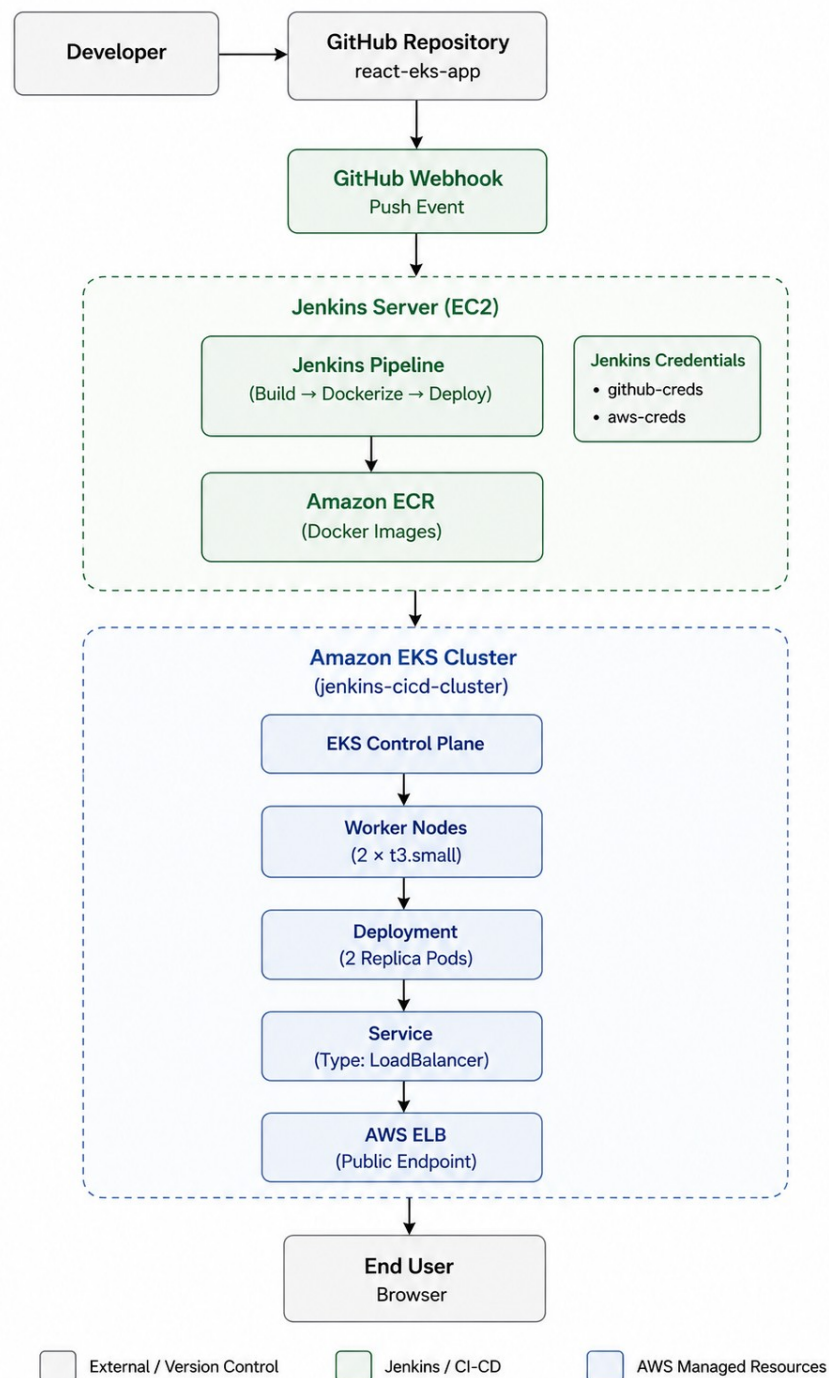
This project implements a complete CI/CD pipeline that automatically builds, containerizes, and deploys a React Vite frontend application to an Amazon EKS (Elastic Kubernetes Service) cluster. Jenkins, hosted on an Amazon EC2 instance, orchestrates the entire workflow — from pulling source code out of a GitHub repository through to updating a live Kubernetes deployment — with no manual intervention required after a developer pushes code.

The goal is to demonstrate a realistic DevOps workflow: a developer pushes a code change to GitHub, a webhook notifies Jenkins, Jenkins builds the application, packages it into a Docker image, pushes that image to Amazon ECR, and finally updates the Kubernetes Deployment running on EKS so the new version becomes live and accessible through a public Load Balancer URL.

Services & Tools Used

- Amazon EC2 — hosts the Jenkins server
- Jenkins — CI/CD automation and pipeline orchestration
- Amazon EKS — managed Kubernetes cluster running the application
- Docker — containerizes the React Vite application
- Kubernetes — deployment, scaling, and service exposure
- GitHub — source code repository and webhook trigger
- AWS IAM — access control and credentials
- AWS CLI — cluster and credential management
- kubectl — Kubernetes cluster interaction
- Amazon ECR — private Docker container registry

2. Architecture



Architecture Explanation

The workflow begins when a developer pushes code to the react-eks-app GitHub repository. A GitHub webhook, configured to fire on every push event, immediately notifies the Jenkins server so that no manual build trigger is needed.

The Jenkins server runs on an EC2 instance and hosts the CI/CD pipeline along with two stored credentials — github-creds for repository access and aws-creds for AWS authentication. When triggered, the Jenkins

pipeline checks out the latest source code, installs dependencies, builds the React Vite application, builds a Docker image from it, and pushes that image to Amazon ECR, which stores the container images.

Jenkins then authenticates to the Amazon EKS cluster (jenkins-cicd-cluster) and updates the Kubernetes Deployment to use the newly pushed image. The EKS cluster consists of a managed control plane and two t3.small worker nodes running two replica pods of the application. A Kubernetes Service of type LoadBalancer exposes these pods to the internet through an AWS Elastic Load Balancer (ELB), which provides a public endpoint. The end user simply opens this URL in a browser to access the live, continuously-updated application.

3. Jenkins Installation Steps

Jenkins was installed on an Ubuntu 24.04 LTS EC2 instance with the following steps:

3.1 Launch EC2 Instance

- Launched an EC2 instance named jenkins-server using the Ubuntu Server 24.04 LTS AMI, with an m7i-flex.large instance type (sufficient headroom for Jenkins plus Docker builds, versus the t3.small used for the EKS worker nodes).
- Created a new key pair (jenkins-key) for SSH access.
- Configured the security group to allow inbound traffic on port 22 (SSH), port 8080 (Jenkins web UI), and port 8081 (test container access).

3.2 Install Java, Jenkins, and Supporting Tools

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install -y jenkins

sudo apt install -y git docker.io
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
```

AWS CLI and kubectl were also installed on the server so pipeline stages could interact with AWS and the EKS cluster directly.

3.3 Start and Verify Jenkins

```
sudo systemctl enable jenkins
sudo systemctl restart jenkins
sudo systemctl status jenkins
```

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Jenkins was then accessed in the browser at `http://<EC2-Public-IP>:8080`, the setup wizard was completed, the suggested plugin bundle was installed along with Docker Pipeline, Amazon ECR, GitHub Integration, and NodeJS plugins, and an administrator account was created.

3.4 Global Tool Configuration

- Git — used the default installation already present on the server.
- NodeJS — added a NodeJS installation with “Install automatically” enabled, using a recent LTS version.
- Docker — configured to use the Docker installation already present on the server.

Jenkins was restarted after configuration to apply the global tool settings.

4. EKS Setup Process

The Amazon EKS cluster was created and verified from a local machine using `eksctl` and the AWS CLI.

4.1 Prerequisites

- Installed AWS CLI, `kubectl`, and `eksctl`.
- Configured AWS credentials using `aws configure` (Access Key ID, Secret Access Key, region, and output format).

4.2 Cluster Creation

```
eksctl create cluster `
  --name jenkins-cicd-cluster `
  --region ap-south-1 `
  --nodegroup-name worker-nodes `
  --node-type t3.small `
  --nodes 2 `
  --nodes-min 2 `
  --nodes-max 3 `
  --managed
```

This provisioned a VPC, subnets, IAM roles, the EKS control plane, and a managed node group of two `t3.small` worker nodes. The process took approximately 15–20 minutes to complete.

4.3 Verification

```
kubectl get nodes
kubectl cluster-info
```

Both worker nodes were confirmed in the Ready state, verifying that the cluster was reachable and functional from the local machine.

5. CI/CD Pipeline Workflow

Jenkins credentials were configured first: a GitHub Personal Access Token stored as github-creds (Username with password) and AWS Access Key/Secret stored as aws-creds (AWS Credentials type). The same IAM identity that created the EKS cluster was used for aws-creds, since that identity automatically holds admin access to the cluster.

A React Vite application was scaffolded using `npm create vite@latest` with the default React template, and pushed to the react-eks-app GitHub repository. A multi-stage Dockerfile was then added — the first stage builds the app using Node.js, and the second stage serves the built static files using Nginx, keeping the final image lightweight. A `.dockerignore` file excludes `node_modules`, `dist`, and Git metadata from the build context.

The image was first built and pushed to Amazon ECR manually to confirm the flow, then a Jenkinsfile was created defining the full automated pipeline with the following stages:

- Checkout Source Code — pulls the latest code from GitHub using github-creds.
- Install Dependencies — runs `npm install`.
- Build React Application — runs `npm run build`.
- Build Docker Image — builds the Docker image, tagged with the Jenkins build number.
- Push Docker Image — authenticates to Amazon ECR and pushes the tagged image.
- Update Kubernetes Deployment — runs `kubectl set image` to point the Deployment at the new image tag.
- Verify Deployment Status — runs `kubectl rollout status` to confirm the rollout completed successfully.

A Jenkins Pipeline job (react-eks-pipeline) was created using “Pipeline script from SCM”, pointing at the GitHub repository and reading the Jenkinsfile from the main branch. The pipeline was first run manually via Build Now and verified to complete all seven stages successfully.

5.1 Continuous Deployment (Webhook Trigger)

To fully automate the pipeline, the “GitHub hook trigger for GITScm polling” option was enabled in the Jenkins job configuration, and a webhook was registered in the GitHub repository settings pointing to `http://<Jenkins-EC2-IP>:8080/github-webhook/` for push events.

With this in place, every git push to the repository automatically triggers the Jenkins pipeline — confirmed by build entries showing “Started by GitHub push” rather than “Started by user” — completing the CI/CD loop from code change to live deployment with zero manual steps.

6. Kubernetes Deployment Process

Kubernetes Deployment and Service manifests were created locally and applied to the EKS cluster to verify the deployment worked manually before wiring it into the Jenkins pipeline.

6.1 Applying the Manifests

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

6.2 Verification

```
kubectl get pods
kubectl get deployment
kubectl get svc react-eks-app-service
```

Both pods reached the Running state with READY 1/1. The Deployment showed 2/2 replicas ready. The Service, of type LoadBalancer, provisioned an AWS Elastic Load Balancer and exposed an EXTERNAL-IP hostname, which was used to access the application live in the browser on port 80.

Once the Jenkinsfile's "Update Kubernetes Deployment" stage was added, this same process — pointing the Deployment at a new image tag and rolling out the update — was fully automated as part of every pipeline run, with kubectl rollout status confirming each rollout completed successfully.

7. Challenges Faced

- Instance type restrictions: The AWS account only permitted certain Free-Tier-eligible instance types. Using t3.medium for the EKS node group initially caused an `AsgInstanceLaunchFailures` error; switching to t3.small resolved it.
- Java version mismatch: The latest Jenkins release requires Java 21 — Java 17 caused Jenkins to fail to start. Installing `openjdk-21-jre` fixed this.
- Expired Jenkins repository key: The Jenkins APT signing key URL initially referenced an outdated/expired key, which had to be updated to the current key before apt install would succeed.
- Missing libatomic1 dependency: The build failed at the Install Dependencies stage because Node.js 26 required the libatomic1 library, which was not present on the Jenkins EC2 server. SSH-ing into the server and running `sudo apt install -y libatomic1`, then re-triggering the build, resolved the failure.
- NodeJS tool auto-install: Leaving "Install automatically" unchecked under the NodeJS Global Tool Configuration caused every pipeline build to fail with "npm: not found." Enabling automatic installation resolved this.
- File extension issues on Windows: Notepad silently appended .txt to files like Dockerfile, .dockerignore, and Jenkinsfile when saving, which had to be corrected by renaming the files before committing.
- Docker permissions: The jenkins and ubuntu users needed to be added to the docker group, followed by a Jenkins restart, before pipeline stages could run Docker commands without permission errors.
- IAM identity mismatch: Using AWS credentials from an IAM identity different from the one that created the EKS cluster would have blocked Jenkins from deploying to the cluster, since only the creating identity gets automatic cluster access. Using the same identity's keys for aws-creds avoided this.
- Public IP changing after instance stop/start: After stopping the Jenkins EC2 instance and restarting it a day later, the instance's public IP address changed, which broke the existing SSH connection and meant the browser URL for Jenkins (`http://<old-ip>:8080`) no longer worked. Reconnecting required looking up and using the new public IP. Since this same IP is also referenced by the GitHub webhook and would silently break every future pipeline trigger after any stop/restart, an Elastic IP was allocated and associated with the instance so the public IP stays fixed regardless of stop/start cycles.

- LoadBalancer provisioning delay: The EXTERNAL-IP for the Kubernetes Service took a few minutes to appear after applying the manifest, as AWS provisions an actual Elastic Load Balancer in the background.

8. Learning Outcomes

- Gained hands-on experience provisioning a managed Kubernetes cluster on AWS using eksctl and understanding EKS control plane and worker node architecture.
- Learned to install, configure, and administer Jenkins, including plugin management, global tool configuration, and credential management.
- Understood how to securely store and use Git and AWS credentials inside Jenkins for pipeline automation.
- Learned to containerize a frontend application using a multi-stage Dockerfile to produce a small, production-ready image.
- Gained experience pushing and managing Docker images in a private Amazon ECR repository.
- Learned to write and apply Kubernetes Deployment and Service manifests, and to expose an application publicly using a LoadBalancer Service.
- Built a complete Jenkins Declarative Pipeline covering checkout, build, containerization, registry push, and Kubernetes deployment stages.
- Understood how GitHub webhooks integrate with Jenkins to achieve true continuous deployment, triggered automatically by every code push.
- Learned the importance of Elastic IPs for keeping infrastructure endpoints (SSH access, Jenkins URL, GitHub webhook target) stable across instance restarts.
- Developed practical troubleshooting skills for real-world DevOps issues such as permission errors, tool version mismatches, missing system libraries, and cloud resource provisioning delays.

9. Kubernetes Configuration

9.1 deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: react-eks-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: react-eks-app
  template:
    metadata:
      labels:
        app: react-eks-app
    spec:
      containers:
```



```
- name: react-eks-app
  image: 626230557743.dkr.ecr.ap-south-1.amazonaws.com/react-eks-app:latest
  ports:
    - containerPort: 80
```

9.2 service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: react-eks-app-service
spec:
  type: LoadBalancer
  selector:
    app: react-eks-app
  ports:
    - port: 80
      targetPort: 80
```

10. Docker Configuration

10.1 Dockerfile

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

10.2 .dockerignore

```
node_modules
dist
.git
.gitignore
README.md
```

11. Jenkinsfile

```
pipeline {
  agent any

  tools {
    nodejs 'Node-js-26'
  }

  environment {
    AWS_ACCOUNT_ID = '626230557743'
    AWS_REGION      = 'ap-south-1'
    ECR_REPO        = 'react-eks-app'
    IMAGE_TAG       = "${env.BUILD_NUMBER}"
    CLUSTER_NAME    = 'jenkins-cicd-cluster'
  }

  stages {

    stage('Checkout Source Code') {
      steps {
        git branch: 'main',
            credentialsId: 'github-creds',
            url: 'https://github.com/snehabiswal006/react-eks-app.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }

    stage('Build React Application') {
      steps {
        sh 'npm run build'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh "docker build -t ${ECR_REPO}:${IMAGE_TAG} ."
      }
    }

    stage('Push Docker Image') {
      steps {
        withCredentials([[class: 'AmazonWebServicesCredentialsBinding',
            credentialsId: 'aws-creds']]) {
          sh ""
        }
      }
    }
  }
}
```

```

        aws ecr get-login-password --region ${AWS_REGION} | docker
login --username AWS --password-stdin ${AWS_ACCOUNT_ID}.dkr.ecr.$
${AWS_REGION}.amazonaws.com
        docker tag ${ECR_REPO}:${IMAGE_TAG} ${AWS_ACCOUNT_ID}.dkr.ecr.
${AWS_REGION}.amazonaws.com/${ECR_REPO}:${IMAGE_TAG}
        docker push ${AWS_ACCOUNT_ID}.dkr.ecr.$
${AWS_REGION}.amazonaws.com/${ECR_REPO}:${IMAGE_TAG}
        """
    }
}
}

stage('Update Kubernetes Deployment') {
    steps {
        withCredentials([[class: 'AmazonWebServicesCredentialsBinding',
credentialsId: 'aws-creds']]) {
            sh """
                aws eks update-kubeconfig --name ${CLUSTER_NAME} --region $
${AWS_REGION}

                kubectl set image deployment/react-eks-app react-eks-app=$
${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/${ECR_REPO}:${IMAGE_TAG}
            """
        }
    }
}

stage('Verify Deployment Status') {
    steps {
        withCredentials([[class: 'AmazonWebServicesCredentialsBinding',
credentialsId: 'aws-creds']]) {
            sh 'kubectl rollout status deployment/react-eks-app'
        }
    }
}
}
}

```